CASE STUDY · AI WORKFLOW DESIGN

From an empty folder to a 61-page, fully cited market report — built with Claude

A repeatable competitor-analysis workflow where AI handles research and judgement, deterministic code handles execution, and every one of 412 claims is traced to a source. This page shows the architecture, the algorithm and the exact prompts, so the method can be reused for any market.

Moustafa Elkoushy
Workflow designer

October 2026
Built and first run

Claude Code + Python
Stack

B2B messaging · Bangladesh
First use case

Business data from the engagement is summarised only; confidential figures are not shown.

01 · RESULTS

What one run produced

A competitor and market analysis for a telecom-messaging company entering Bangladesh, delivered as a branded PDF for investors and partners.

9
competitors profiled in depth, plus 30 in a market overview table

192
unique sources, each with URL and access date

412
factual claims — every one linked to a source

0
uncited claims allowed — a validator blocks the PDF otherwise

7
Python tools written and tested along the way

9
research agents run in parallel, one per competitor

4
human checkpoints where the owner approves before work continues

~50
claims from older internal files checked; 15 corrected or dropped

THE POINT

The value isn't that AI wrote a report. It's that the report can be **trusted and re-run**: the same structure, the same evidence rules and the same branded output every quarter, with a person approving each stage.

02 · THE PROBLEM

Why "just ask the AI" doesn't work for research

ACCURACY COMPOUNDS

Small errors multiply

If each step is 90% accurate, five steps in a row succeed only 59% of the time. A long research task done in one go drifts.

UNSOURCED CLAIMS

Plausible isn't verified

Language models produce confident text. Without a hard rule, a report mixes facts, company self-claims and guesses.

ONE-OFF OUTPUT

Nothing repeatable

A chat answer can't be re-run next quarter with the same structure, branding and quality bar.

The design answer

Split the work by what each part is good at. **AI decides and writes** — which competitors matter, what a page says, what the evidence means. **Code executes** — fetching sites, scoring speed, checking citations, merging data, drawing charts, laying out the PDF. **A person approves** at fixed checkpoints.

03 · ARCHITECTURE

Three layers: Workflows, Agents, Tools

The project follows a "WAT" structure. Instructions live in plain-language SOPs, the AI agent orchestrates, and Python scripts do the deterministic work.



Figure 1 — Probabilistic work (deciding, writing) sits in the agent; anything that must be exact sits in tested code.

04 · THE ALGORITHM

End-to-end flow, with human checkpoints

The workflow runs in phases. Build phases happen once; run phases repeat every time the report is refreshed. Diamonds are checkpoints — the agent stops until the owner approves.

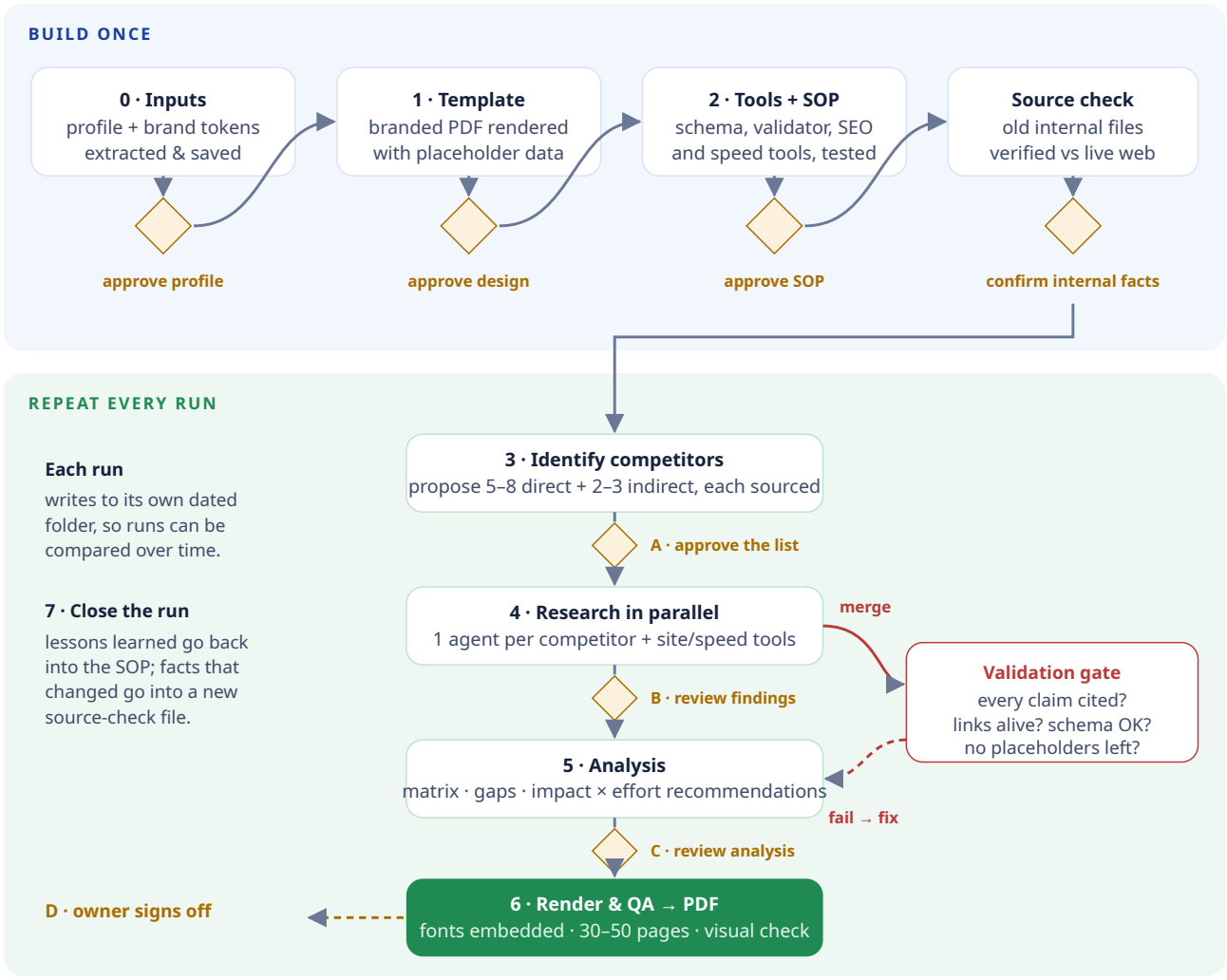


Figure 2 — Build phases (top) run once. Run phases (bottom) repeat each quarter, with the validation gate blocking the PDF until the data is clean.

The same algorithm, as pseudocode

```
run_report(market, date):
    profile  ← read("business-profile.md")             # owner-confirmed facts
    brand    ← read("brand/brand.json")
    known    ← latest("research/source_check_*.md")    # verified facts + known-
false claims

    list     ← agent.propose_competitors(profile, search)    # 5–8 direct, 2–3
indirect
    checkpoint A: owner approves list

    parallel for c in list:                                  # one sub-agent each
        files[c] ← agent.research(c, brief, known[c])     # JSON matching the
schema
    seo   ← tool.seo_snapshot(all_sites)
    speed ← tool.pagespeed(all_sites)

    report ← tool.merge(base, files, speed)           # dedupe + renumber
sources
    repeat until tool.validate(report) = PASS:
        fix(report)                                   # drop or re-source
failing claims
    checkpoint B: owner reviews findings

    report.analysis ← agent.analyse(report)           # matrix, gaps,
recommendations
    checkpoint C: owner reviews analysis

    pdf ← tool.render(report, brand)                  # charts + template +
WeasyPrint
    qa(pdf)                                           # fonts, page count,
visual check
    checkpoint D: owner signs off
    update_sop(lessons_learned)
```

05 · RESEARCH ENGINE

Fan out, then funnel through one gate

Research is the slowest step, so it runs in parallel: one Claude sub-agent per competitor, all following the same written brief. Their outputs are funnelled through deterministic merge and validation before anything is rendered.

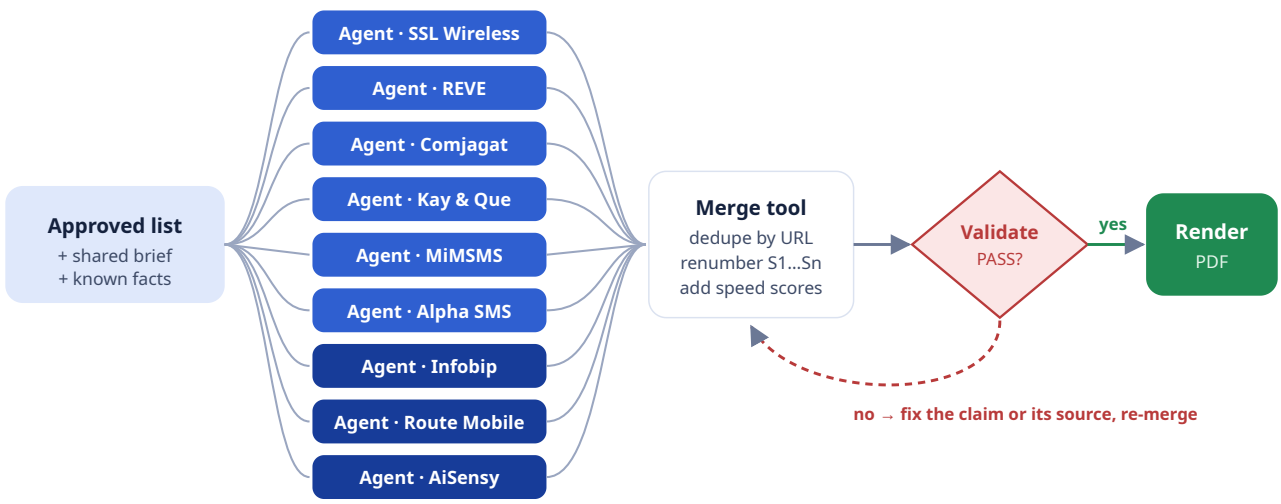
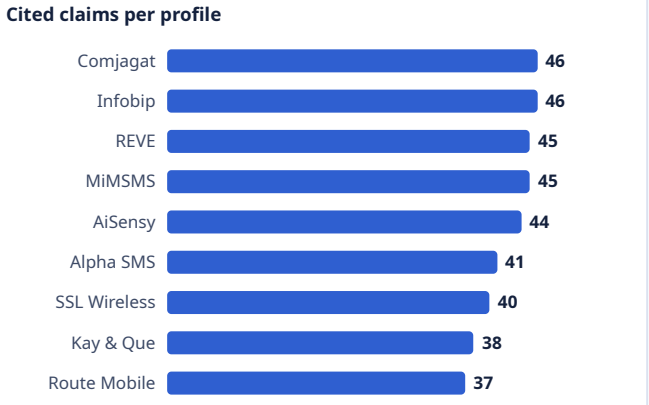
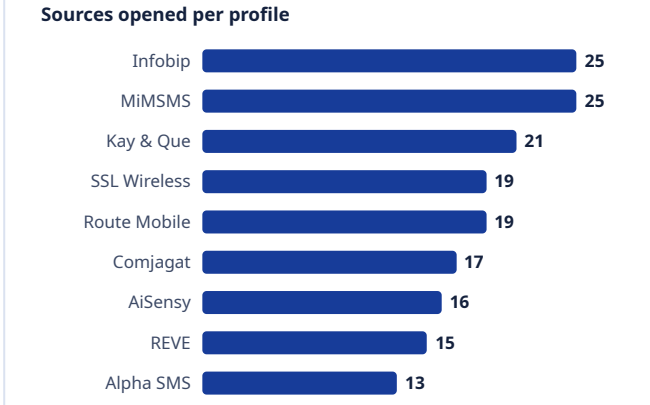


Figure 3 — Nine agents ran at once in the first run. Each returned one JSON file; the merge tool combined 192 unique sources from them.

Evidence collected per competitor

Two separate charts rather than one with two scales: claims and sources measure different things.





Counts from the first run's research files. Sources are deduplicated across profiles in the final report (192 unique).

06 · BUILDING TRUST INTO THE OUTPUT

The validation gate and the evidence rules

The single most important design decision: the PDF **cannot be rendered** unless a deterministic validator passes. The AI is told the rules; the code enforces them.

What the validator checks

1. **Schema**
   Every field matches a JSON schema; every claim carries at least one source id.
2. **Citations**
   Every cited id exists; every external source has a URL and an ISO access date.
3. **Completeness**
   The comparison matrix scores every dimension for every company.
4. **No placeholders**
   Production runs fail on "Sample" or "TODO" text and example URLs.
5. **Forbidden terms**
   Names that must never appear in a report are blocked.
6. **Live links**
   404s and dead domains fail; bot-blocked pages are flagged for a manual check.

Evidence rules every agent follows

FACT	Only cite pages actually opened. A search snippet is marked as one.
CLAIM	Company statements are labelled: "claims 60+ banks", not "serves".
INFERENCE	Reasoning is marked "likely" or "suggests" — never stated as fact.
GAP	No evidence → "not publicly disclosed", citing where it was checked.

Old data is treated as unverified

Before research, the internal files (a previous study, sales decks) were checked against live sources by three parallel agents. About 50 claims were marked confirmed, outdated, contradicted or unverifiable. Examples of what that caught:

OUTDATED

A competitor's client count had changed (45+ banks → 60+ today).

UNSUPPORTED

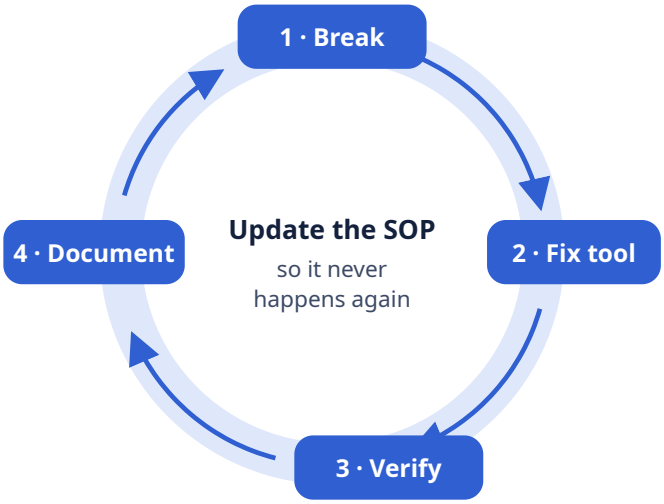
A "80% market share" figure had no public source and was dropped.

NEW

Regulator notices revealed 71 licence cancellations, not the 63 on file.

07 · SELF-IMPROVEMENT LOOP

Every failure makes the system stronger



Real failures from the first build, each fixed in the tool and recorded in the workflow's "lessons learned":

WHAT BROKE	FIX
A website request hung forever	Hard wall-clock deadline + one retry
A regex took 5 min of CPU	Replaced with a simple line parser
Brand font silently missing in PDF	Template escaped CSS quotes; marked safe, now checked
An everyday phrase flagged as a banned company name	Case-sensitive, whole-word matching
Free API quota exhausted	Restricted API key in `.env`

08 · STACK

Tools and why they were chosen

LAYER	CHOICE	WHY
Agent	Claude Code (plan mode, sub-agents)	Plans before acting, asks questions in one batch, runs parallel research agents.
Research	Web search + page fetch	Free; every claim points to a page that was actually opened.
Site data	requests + BeautifulSoup; Google PageSpeed API	Same checks on every site, every run; free speed and SEO scores.
Data contract	JSON Schema	Agents' output has one fixed shape, so code can merge and check it.
PDF	Jinja2 + WeasyPrint (HTML/CSS)	Running headers, page numbers, clickable citations, brand fonts embedded.
Charts	matplotlib → SVG	Brand colours and fonts; sharp at any zoom.

Seven tools in total: brand extractor, SEO snapshot, PageSpeed, validator, merge, charts and renderer — about 1,250 lines of Python, plus a 265-line schema and a 120-line SOP.

09 · PROMPT PLAYBOOK

The prompts, in order

These are the prompts to build the same workflow from zero in Claude Code. Replace the `[yellow parts]` with your own details. Send them one at a time and review each result before sending the next.

0 Set the rules of the project

Save as CLAUDE.md in an empty folder before anything else

```
You work inside the WAT framework (Workflows, Agents, Tools).
- Workflows: Markdown SOPs in workflows/ — objective, inputs, tools, outputs, edge
cases.
- Agent (you): read the workflow, run tools in order, handle failures, ask when
unsure.
- Tools: Python scripts in tools/ that do the deterministic work. Secrets only in
.env.

Rules:
1. Look for an existing tool before writing a new one.
2. When something fails: read the full error, fix the tool, re-test, and record what
you
   learned in the workflow. Ask me before re-running anything that costs money.
3. Don't overwrite a workflow without asking me.
4. Temporary files go in .tmp/ and are disposable. Final outputs go to [where I keep
deliverables].
```

Why: this file is loaded into every session, so the split between AI judgement and code execution holds for the whole project.

1 Initialise the project

First message in the folder

```
I just added a CLAUDE.md that explains how I want you to work in this project.
Initialise the project and set everything up (folders, .gitignore, .env.example,
requirements.txt, a Python virtual environment). Ask me any questions you have
first.
```

2 Describe the workflow and ask for a plan

Use plan mode — no code yet

```
I want to build a REPEATABLE competitor analysis and market research workflow.
The final output of each run is a fully branded PDF report.

Inputs I will provide:
- Business info about my company. Save it to business-profile.md so every run reuses
it.
- Brand guidelines (colours, fonts, tone) and a logo. The entire PDF must follow
them:
  cover, headers, charts, tables, footer.

What the workflow should do:
1. Read business-profile.md.
2. Identify [5–8] direct and [2–3] indirect competitors in [market / region].
3. For each competitor research: positioning and messaging, pricing, product
features,
   website and SEO, content and social, customer reviews (praise and complaints),
   recent moves (launches, partnerships, expansion).
4. Compare them against my business in a matrix.
5. Identify what works for them, where they are weak, market gaps, and concrete
   improvements for my business, prioritised by impact and effort.
6. Every claim must cite a source URL and access date.

Report structure: Cover → Executive summary → Market overview → Competitor profiles
→
Comparison matrix → Opportunities & gaps → Recommendations → Sources

Before planning, ask me ALL your clarifying questions in one batch. Then give me a
plan
with: phases, files and folders you will create, the PDF library you will use and
why,
and a checkpoint after each phase where I review before you continue.
Don't write code until I approve.
```

Why: "all questions in one batch" and "a checkpoint after each phase" are what keep a long build under control.

3 Give the business profile

Paste after answering Claude's questions

```
Here is my business profile. Save it to business-profile.md exactly as written.
Then tell me what in it changes your plan, and list anything that looks inconsistent
or needs a source.

[paste: company overview, what you sell, customers and segments, what makes you
different, go-to-market channels, market and region, known competitors,
companies to leave out, open questions]
```

4 Hand over the brand

Put the files in brand/ first

```
I put my brand guidelines, logo and fonts in brand/. Extract the colours, fonts,
type scale, logo rules, motif and tone of voice into brand/brand.json.
Show me what you found and flag any rule that conflicts with this report
(for example a rule about naming competitors) before you use it.
```

5 Design first, with placeholder data

Approve the look before any research is spent

```
Build the branded PDF template now, using placeholder data only — no research yet.
Cover every section type: cover, contents, executive summary, a competitor profile,
the comparison matrix, charts, recommendation table and sources.
Render a sample PDF, check that the brand fonts are embedded and there are no blank
pages, and show me the result.
```

6 Verify the internal files

Old studies and decks are useful — and often wrong

```
I added files related to the business in [folder]. Some information may no longer
be valid, so check everything. Mark each external claim as confirmed, outdated,
contradicted or unverifiable, with the source link. Internal facts you can't check
online: list the conflicts and ask me. Don't update the business profile until I
answer.
```

7 Build and test the research tools

Deterministic parts become scripts

```
Build the research tools and the workflow SOP:
- a JSON schema for the report data (every claim must carry source ids),
- a validator that blocks rendering if any claim is unsourced, a link is dead,
  or placeholder text is left,
- a website snapshot tool and a PageSpeed tool,
- workflows/[name].md with steps, checkpoints, evidence rules and edge cases.
Test every tool on my own website first and show me the results.
```

8 Choose the competitors

Checkpoint A

```
Start the first run. Propose [5–8] direct and [2–3] indirect competitors with a one-line
rationale and a source for each. Search for new entrants I might not know about.
Put everyone else in a short "rest of the market" table. Wait for my approval.
```

9 Research in parallel

Checkpoint B

```
Research each approved competitor in parallel, one sub-agent per company, all with
the same written brief: only cite pages actually opened, label company self-claims
as claims, label inference, and say "not publicly disclosed" when there is no
evidence.
Run the website and speed tools on every site. Merge everything, run the validator
until it passes, then show me the findings before writing any analysis.
```

10 Write the analysis

Checkpoint C

```
Write the analysis from the approved evidence only: score the comparison matrix 1–5,
draw a positioning map, list what works for competitors, their weak spots and the
market gaps, and give ranked recommendations with impact, effort and timeframe.
Write the executive summary last. Every claim must cite the evidence it rests on.
Be honest about my own weaknesses — the readers are [investors / leadership].
```

11 **Polish and sign off**

Checkpoint D

```
Render the final PDF. Keep it to [30–50] pages, check fonts are embedded, no blank
pages, every section filled, and look at every page as an image before you hand it
over.
Then add what we learned in this run to the workflow's "lessons learned" section.
```

12 **Re-run next quarter**

The payoff of building it as a workflow

```
Run workflows/[name].md for [month year]. Re-check the facts in the latest source
check,
compare against the previous run in output/, and add a short "what changed since
last
time" section. Stop at every checkpoint as usual.
```

Habits that made the difference

Plan before code

Plan mode plus "don't write code until I approve" kept the build in the right order.

Design before research

Approving a placeholder PDF first meant no research effort was wasted on a layout that would change.

Delegate a decision explicitly

"You take the decision" works — Claude then states the choice and its reason in one line.

Never paste secrets into chat

Put API keys straight into `.env`, and restrict each key to the one API it needs.

10 · REUSE

Adapting it to another market

Only three inputs change: `business-profile.md`, the files in `brand/`, and the market named in prompt 8. The schema, tools, validator, template and SOP are reused as they are. The same pattern — agent decides, code executes, human approves, validator gates — fits any research deliverable where trust matters: due diligence, supplier screening, grant landscapes or policy monitoring.